Simple Types

Concepts of Programming Languages

CAS CS 320

Outline

- Have a high-level discussion of **type theory** in general
- Introduce and analyze the **simply-typed lambda calculus** (STLC)
- Demo an **implementation** of the STLC

Type Theory

What is a Type?

```
let f : int -> int = ...
```

What is a Type?

```
let f : int -> int = ...
```

Who knows...

What is a Type?

```
let f : int -> int = ...
```

Who knows...

A **type** is an *syntactic object* given to an expression to describe its (semantic) behavior

What is a Type?

```
let f : int -> int = ...
```

Who knows...

A **type** is an *syntactic object* given to an expression to describe its (semantic) behavior

This description can be used to *restrict* the use of the expression *within* a program

What is a Type?

```
let f : int -> int = ...
```

Who knows...

A **type** is an *syntactic object* given to an expression to describe its (semantic) behavior

This description can be used to *restrict* the use of the expression *within* a program

Types help us delineate "well-behaved" programs

Trade-offs

$$(\lambda x\,.\,xx)(\lambda x\,.\,xx)$$

lambda term called Ω

Trade-offs

$$(\lambda x\,.\,xx)(\lambda x\,.\,xx)$$

lambda term called Ω

Types are *restrictive*. They tells us what we *can't* do in our programs

Trade-offs

$$(\lambda x\,.\,xx)(\lambda x\,.\,xx)$$

lambda term called Ω

Types are *restrictive*. They tells us what we *can't* do in our programs

But types are *safe*. They make sure we don't do dumb things in our program

Trade-offs

$$(\lambda x\,.\,xx)(\lambda x\,.\,xx)$$

lambda term called Ω

Types are *restrictive*. They tells us what we *can't* do in our programs

But types are *safe*. They make sure we don't do dumb things in our program

The goal is to balance:

Trade-offs

$$(\lambda x\,.\,xx)(\lambda x\,.\,xx)$$

lambda term called Ω

Types are *restrictive*. They tells us what we *can't* do in our programs

But types are *safe*. They make sure we don't do dumb things in our program

The goal is to balance:

- Simplicity/Usability

Trade-offs

$$(\lambda x\,.\,xx)(\lambda x\,.\,xx)$$

lambda term called Ω

Types are *restrictive*. They tells us what we *can't* do in our programs

But types are *safe*. They make sure we don't do dumb things in our program

The goal is to balance:

- Simplicity/Usability
- Expressivity

Trade-offs

$$(\lambda x\,.\,xx)(\lambda x\,.\,xx)$$

lambda term called Ω

Types are *restrictive*. They tells us what we *can't* do in our programs

But types are *safe*. They make sure we don't do dumb things in our program

The goal is to balance:

- Simplicity/Usability
- Expressivity
- Safety/Theoretical Guarantees

OCaml

```
# let big_omega =
    let little_omega x = x x in
    little_omega little_omega;;
Error: This expression has type 'a -> 'b
       but an expression was expected of type 'a
       The type variable 'a occurs inside 'a -> 'b
```

OCaml

```
# let big_omega =
    let little_omega x = x x in
    little_omega little_omega;;
Error: This expression has type 'a -> 'b
       but an expression was expected of type 'a
       The type variable 'a occurs inside 'a -> 'b
```

OCaml tells us when we try to write ill-behaved program

OCaml

```
# let big_omega =
    let little_omega x = x x in
    little_omega little_omega;;
Error: This expression has type 'a -> 'b
       but an expression was expected of type 'a
       The type variable 'a occurs inside 'a -> 'b
```

OCaml tells us when we try to write ill-behaved program

But OCaml also has strong *type inference* and *polymorphism*

OCaml

```
# let big_omega =
    let little_omega x = x x in
    little_omega little_omega;;
Error: This expression has type 'a -> 'b
       but an expression was expected of type 'a
       The type variable 'a occurs inside 'a -> 'b
```

OCaml tells us when we try to write ill-behaved program

But OCaml also has strong *type inference* and *polymorphism*

More expressive means more complex, designing PLs is finding the balance

Simply-Typed Lambda Calculus

Typing Judgments

Syntax (STLC)

Typing (STLC)

Example

$$\{x : \bot \to \bot\ , y : \bot\ \} \vdash \lambda z\,.\,z(xy) : (\ \bot \to \bot\ ) \to \bot$$

Example

$$\emptyset \vdash \lambda x \,.\, xx :$$

Type Annotations

Aside: Church vs. Curry Typing

```
fun x -> x

fun (x : unit) -> x
```

Aside: Church vs. Curry Typing

```
fun x -> x

fun (x : unit) -> x
```

What is the type of the first expression? How about the second?

Aside: Church vs. Curry Typing

```
fun x -> x

fun (x : unit) -> x
```

What is the type of the first expression? How about the second?

In **Curry-style typing,** the type of an expression is *extrinsic*, the expression is just an expression in the lambda calculus

Aside: Church vs. Curry Typing

```
fun x -> x

fun (x : unit) -> x
```

What is the type of the first expression? How about the second?

In **Curry-style typing**, the type of an expression is *extrinsic*, the expression is just an expression in the lambda calculus

In **Church-style typing**, it's *intrinsic*, built into the expression and the semantics

Aside: Church vs. Curry Typing

```
fun x -> x

fun (x : unit) -> x
```

What is the type of the first expression? How about the second?

In **Curry-style typing**, the type of an expression is *extrinsic*, the expression is just an expression in the lambda calculus

In **Church-style typing**, it's *intrinsic*, built into the expression and the semantics

(using Curry-style typing is not the same as having polymorphism)

Uniqueness of Types

Uniqueness of Types

Lemma. If $\Gamma \vdash e : \tau_1$ and $\Gamma \vdash e : \tau_2$ then $\tau_1 = \tau_2$

Uniqueness of Types

Lemma. If $\Gamma \vdash e : \tau_1$ and $\Gamma \vdash e : \tau_2$ then $\tau_1 = \tau_2$

Proof. The rough idea is to do induction *on the expressions themselves*

Uniqueness of Types

Lemma. If $\Gamma \vdash e : \tau_1$ and $\Gamma \vdash e : \tau_2$ then $\tau_1 = \tau_2$

Proof. The rough idea is to do induction *on the expressions themselves*

Every expression has a *unique type* in STLC with Church-style typing

Uniqueness of Types

Lemma. If $\Gamma \vdash e : \tau_1$ and $\Gamma \vdash e : \tau_2$ then $\tau_1 = \tau_2$

Proof. The rough idea is to do induction *on the expressions themselves*

Every expression has a *unique type* in STLC with Church-style typing

In particular, the function `type_of` is well-defined

Recall: Semantics (STLC)

$$(1)\ \frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$(2)\ \frac{e_1 \Downarrow \lambda x\,.\,e \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}$$

Recall: Semantics (STLC)

$$(1)\ \frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$(2)\ \frac{e_1 \Downarrow \lambda x\,.\,e \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}$$

We can use any semantics we want

Recall: Semantics (STLC)

$$(1)\ \frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$(2)\ \frac{e_1 \Downarrow \lambda x\,.\,e \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}$$

We can use any semantics we want

This is part of the point: Type-checking only determines *whether* we evaluate the program (whether it makes sense to)

Recall: Semantics (STLC)

$$(1)\ \frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$(2)\ \frac{e_1 \Downarrow \lambda x\,.\,e \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}$$

We can use any semantics we want

This is part of the point: Type-checking only determines *whether* we evaluate the program (whether it makes sense to)

It doesn't determine **how** we evaluate the program

Practice Problem

$$\frac{(x:\tau) \in \Gamma}{\Gamma \vdash x : \tau} \qquad \frac{\Gamma, x:\tau \vdash e : \tau'}{\Gamma \vdash \lambda x^{\tau}.\, e : \tau \to \tau'}$$

$$\frac{\Gamma \vdash e_1 : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 e_2 : \tau'}$$

$\cdot \vdash \lambda f^{\bot \to \bot}.\, \lambda x^{\bot}.\, fx : (\bot \to \bot) \to \bot \to \bot$

Give a derivation for the above judgment

Answer

$$\cdot \vdash \lambda f^{\top \to \top} . \lambda x^{\top} . fx : (\top \to \top) \to \top \to \top$$

How do we know if we've defined a "good" programming language?

Type Safety

Type Safety

Thm. If $\cdot \vdash e : \tau$ and there is a value v such that $\langle \varnothing, e \rangle \Downarrow v$ then $\cdot \vdash e(v) : \tau$

Type Safety

> **Thm.** If $\cdot \vdash e : \tau$ and there is a value v such that $\langle \varnothing, e \rangle \Downarrow v$ then $\cdot \vdash e(v) : \tau$

With small-step semantics, we can give a finer-grained analysis:

Type Safety

> **Thm.** If $\cdot \vdash e : \tau$ and there is a value v such that $\langle \varnothing, e \rangle \Downarrow v$ then $\cdot \vdash e(v) : \tau$

With small-step semantics, we can give a finer-grained analysis:

> **Thm.** If $\cdot \vdash e : \tau$, then
>
> » *(progress)* either e is a value or there is an e' such that $e \longrightarrow e'$
>
> » *(preservation)* If $\cdot \vdash e : \tau$ and $e \longrightarrow e'$ then $\cdot \vdash e' : \tau$

Type Safety

> **Thm.** If $\cdot \vdash e : \tau$ and there is a value v such that $\langle \varnothing, e \rangle \Downarrow v$ then $\cdot \vdash e(v) : \tau$

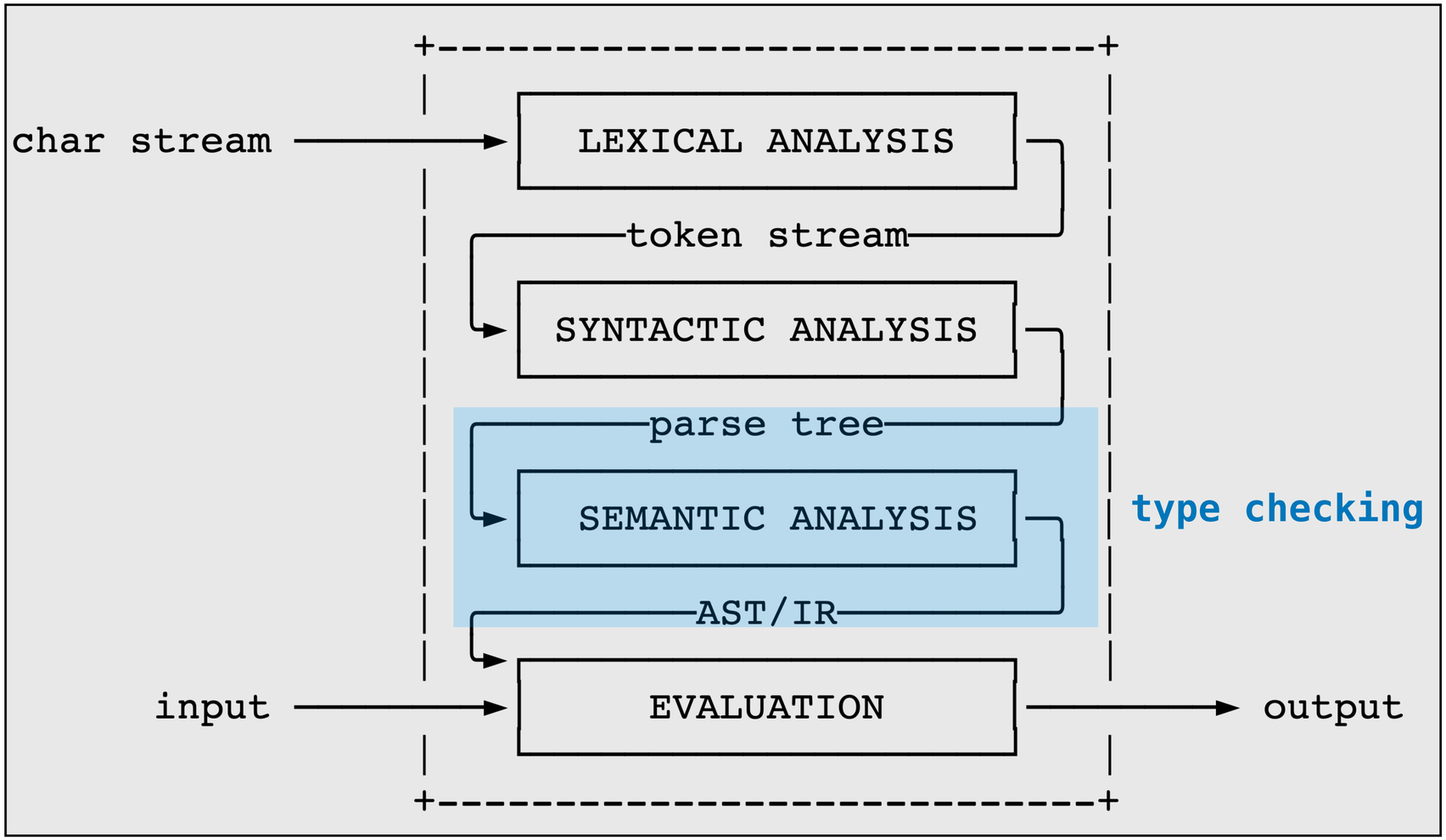
With small-step semantics, we can give a finer-grained analysis:

> **Thm.** If $\cdot \vdash e : \tau$, then
>
> » *(progress)* either e is a value or there is an e' such that $e \longrightarrow e'$
>
> » *(preservation)* If $\cdot \vdash e : \tau$ and $e \longrightarrow e'$ then $\cdot \vdash e' : \tau$

These results are *fundamental*. They tell us that our PL is well-behaved (it's a "good" PL)

Type Checking

The Picture


char stream
LEXICAL ANALYSIS
token stream
SYNTACTIC ANALYSIS
parse tree
SEMANTIC ANALYSIS
type checking
AST/IR
input
EVALUATION
output

Type Checking vs. Type Inference

```
type_check : expr -> ty -> bool

type_of : expr -> ty option
```

Type Checking vs. Type Inference

```
type_check : expr -> ty -> bool

type_of : expr -> ty option
```

Type checking the problem of determining whether a given expression is a given type

Type Checking vs. Type Inference

```
type_check : expr -> ty -> bool

type_of : expr -> ty option
```

Type checking the problem of determining whether a given expression is a given type

Type inference is the problem of *synthesizing* a type for a given expression, if possible

Type Checking vs. Type Inference

```
type_check : expr -> ty -> bool

type_of : expr -> ty option
```

Type checking the problem of determining whether a given expression is a given type

Type inference is the problem of *synthesizing* a type for a given expression, if possible

Theoretically, these two problems can be very different

Type Checking vs. Type Inference

```
type_check : expr -> ty -> bool

type_of : expr -> ty option
```

Type checking the problem of determining whether a given expression is a given type

Type inference is the problem of *synthesizing* a type for a given expression, if possible

Theoretically, these two problems can be very different

For STLC, they are both easy

The One Issue

$$\frac{\Gamma \vdash e_1 : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 e_2 : \tau'}$$

The One Issue

$$\frac{\Gamma \vdash e_1 : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 e_2 : \tau'}$$

How do we turn this into a type-checking procedure?

The One Issue

$$\frac{\Gamma \vdash e_1 : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 e_2 : \tau'}$$

How do we turn this into a type-checking procedure?

It seems like we need to do *some* amount of inference because it's not immediately clear what type we should check e_1 to be

The One Issue

$$\frac{\Gamma \vdash e_1 : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 e_2 : \tau'}$$

How do we turn this into a type-checking procedure?

It seems like we need to do *some* amount of inference because it's not immediately clear what type we should check e_1 to be

Aside: If you're interested there is a way of *combining* checking and inference in what's called *bidirectional type checking*

The One Issue

$$\frac{\Gamma \vdash e_1 : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 e_2 : \tau'}$$

How do we turn this into a type-checking procedure?

It seems like we need to do *some* amount of inference because it's not immediately clear what type we should check e_1 to be

Aside: If you're interested there is a way of *combining* checking and inference in what's called *bidirectional type checking*

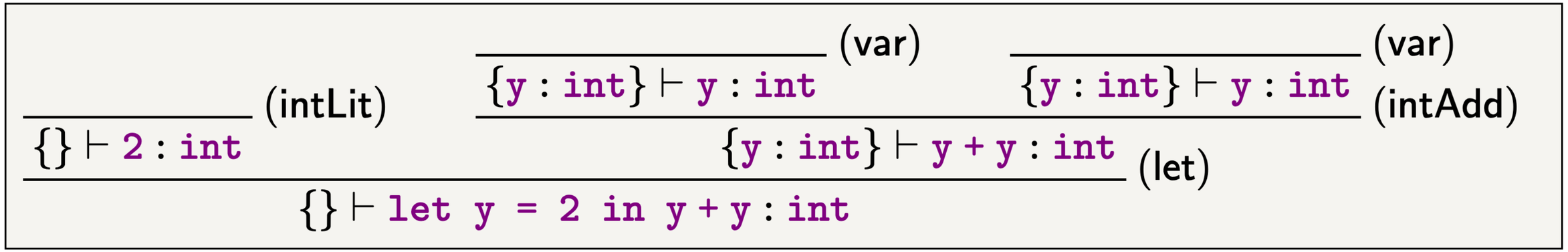
Our solution: We'll just use type inference

demo
(STLC)

Induction over Derivations

The Key Idea

The Key Idea



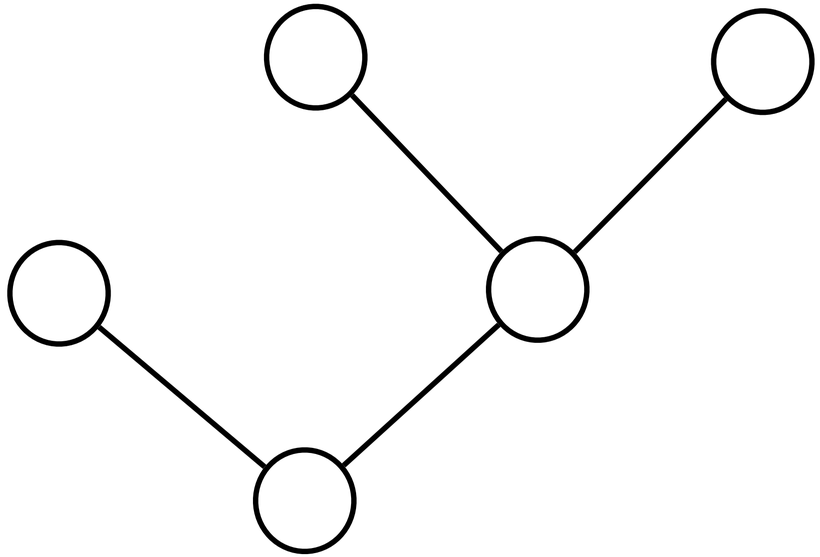
Derivations are *trees*

The Key Idea

$$\dfrac{\dfrac{}{\{\} \vdash \texttt{2} : \texttt{int}}\,(\text{intLit}) \qquad \dfrac{\dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var}) \qquad \dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var})}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} + \texttt{y} : \texttt{int}}\,(\text{intAdd})}{\{\} \vdash \texttt{let y = 2 in y + y} : \texttt{int}}\,(\text{let})$$

Derivations are *trees*

We can prove things about trees using induction



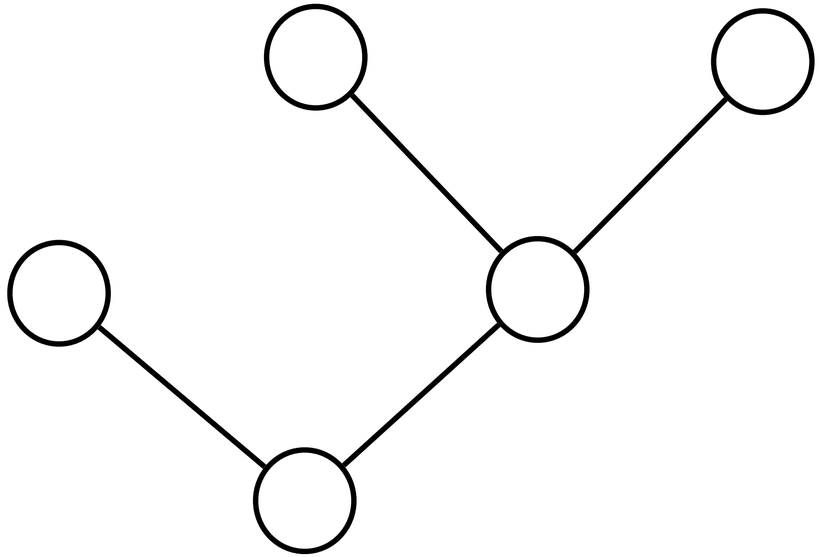
The Key Idea

$$\dfrac{\dfrac{}{\{\} \vdash \texttt{2} : \texttt{int}}\,(\text{intLit}) \qquad \dfrac{\dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var}) \qquad \dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var})}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} + \texttt{y} : \texttt{int}}\,(\text{intAdd})}{\{\} \vdash \texttt{let y = 2 in y + y} : \texttt{int}}\,(\text{let})$$

Derivations are *trees*

We can prove things about trees using induction

We can prove things about *derivable judgments* using induction



The Key Idea

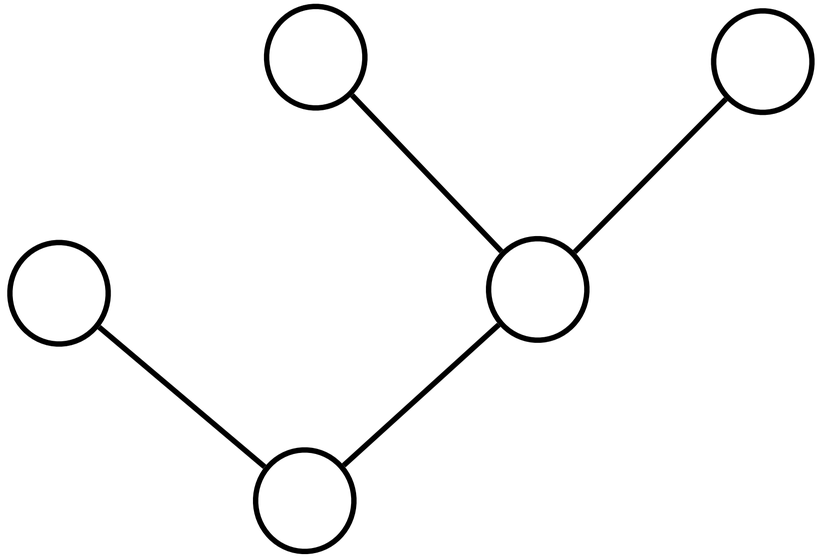
$$\dfrac{\dfrac{}{\{\} \vdash \texttt{2} : \texttt{int}}\,(\text{intLit}) \qquad \dfrac{\dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var}) \qquad \dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var})}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} + \texttt{y} : \texttt{int}}\,(\text{intAdd})}{\{\} \vdash \texttt{let y = 2 in y + y} : \texttt{int}}\,(\text{let})$$

Derivations are *trees*

We can prove things about trees using induction

We can prove things about *derivable judgments* using induction

Important: Every derivable judgment corresponds to a derivation



Warm-up: Well-Scopedness

Warm-up: Well-Scopedness

An expression e is **well-scoped** with respect to a context Γ if $x \in FV(e)$ implies x appears in Γ

Warm-up: Well-Scopedness

An expression e is **well-scoped** with respect to a context Γ if $x \in FV(e)$ implies x appears in Γ

> **Theorem.** If e is well-typed in Γ, then e is well-scoped

Warm-up: Well-Scopedness

An expression e is **well-scoped** with respect to a context Γ if $x \in FV(e)$ implies x appears in Γ

> **Theorem.** If e is well-typed in Γ, then e is well-scoped

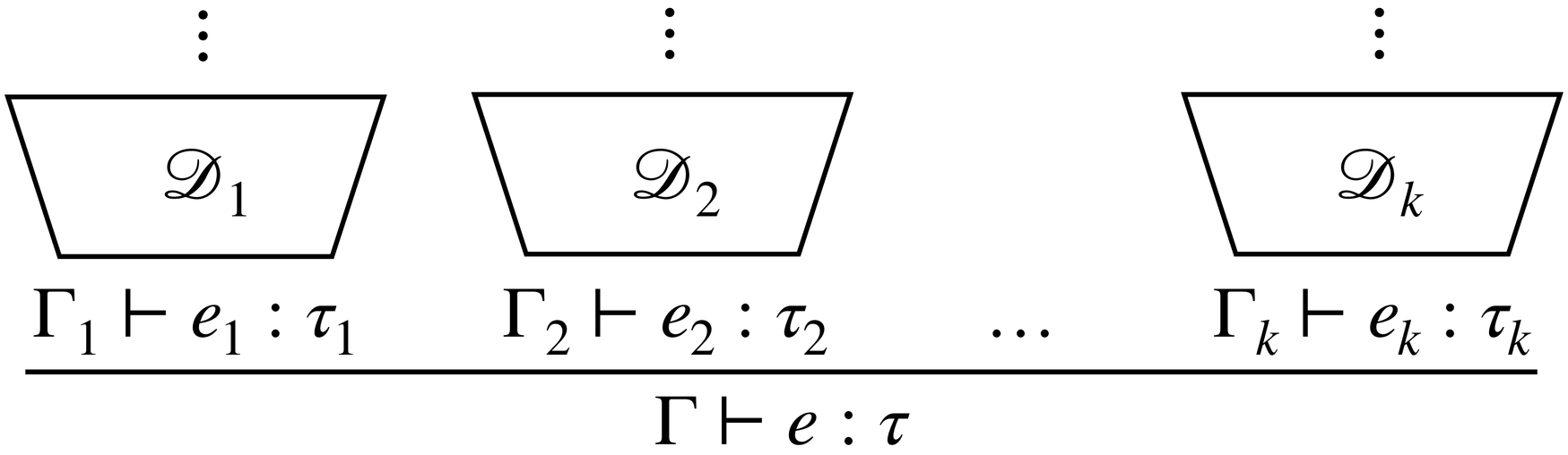
Proof. By induction on *derivations*

Base Case: Axioms

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau}\ \texttt{variable}$$

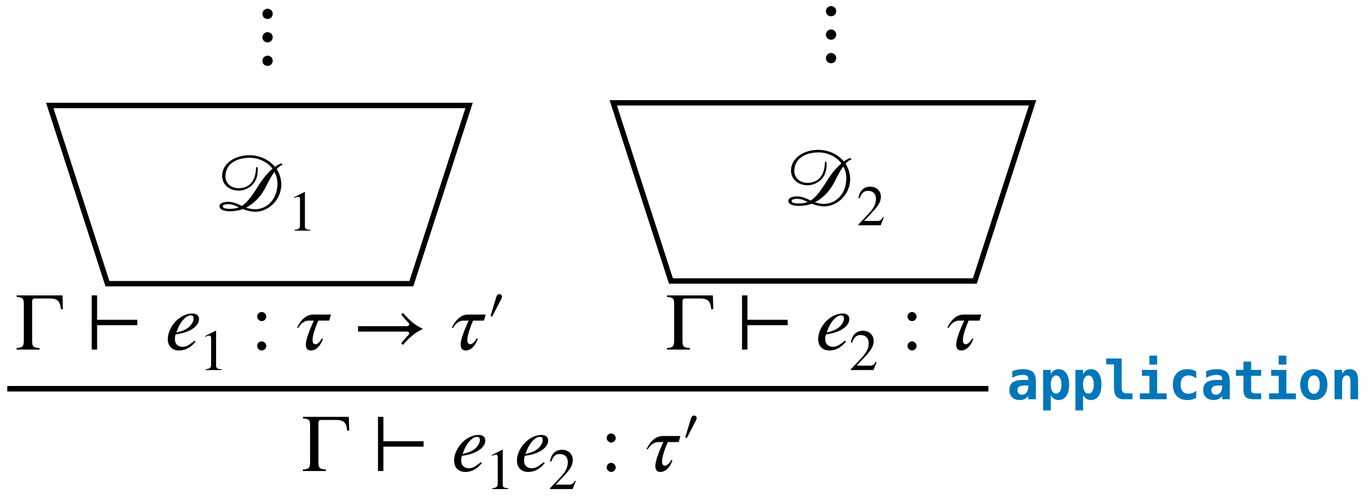
We need to show that expressions typed using just axioms satisfy well-scopedness

Inductive Hypothesis



If $e_1, \ldots, e_k$ are well-scoped (because they are typeable in the each of their contexts)

Inductive Step 1: Application



What if the last rule I applied was application?

Inductive Step 2: Abstraction

$$\dfrac{\begin{array}{c}\vdots \\ \mathscr{D} \\ \Gamma, x : \tau \vdash e : \tau'\end{array}}{\Gamma \vdash \lambda x^{\tau} . e : \tau \to \tau'}\ \textbf{abstraction}$$

What if the last rule I applied was abstraction?

Summary

Type systems delineate well-behaved expressions

We prove **meta-theoretic** properties about types systems to demonstrate this